

Home Assistant Schichtplaner YAML Master Repository V2.0

Erstellt: 23.05.2026 19:56

Diese Version kombiniert: die ursprüngliche YAML-Code-Sammlung die Gemini-Repository-Erweiterungen Legacy-Versionen aktuelle Produktionsversionen Fehleranalysen Frontend-/Backend-Vergleiche Statistiksysteme Dashboard-Konfigurationen Ziel war eine vollständige technische YAML-Referenz des gesamten Schichtplaner-Projekts.

1. Hauptsript — Schichtplaner Final

Zentrales Backend-Sript für die Erstellung aller Kalendereinträge. Verantwortlich für Routing, Zeitlogik und Kalenderschreibzugriffe.

```
alias: "Schichtplaner: Final"

sequence:

  - action: calendar.create_event

    target:
      entity_id: >-
        {% if person == 'Sascha' %}
          calendar.sascha

        {% elif person == 'Sabine' %}
          calendar.sabine

        {% else %}
          calendar.familie
        {% endif %}

    data:

      summary: "{{ schicht }}"

      start_date_time: "{{ tag }} {{ start_zeit }}"
      end_date_time: "{{ tag }} {{ end_zeit }}"
```

2. Legacy Zeitraumplaner v1.2 (Veraltet)

Diese Version enthielt noch das Geisterbuchungsproblem, da blind auf den globalen Dropdown-Helfer zugegriffen wurde. ■filecite■turn4file0■

```
alias: "Schichtplaner: Zeitraum planen (Legacy)"

sequence:

  - variables:

      t_start: "{{ states('input_datetime.ziel_datum') }}"
      t_end: "{{ states('input_datetime.ziel_datum_ende') }}"

      v_person: "{{ states('input_select.planung_person') }}"

      # Kritischer Fehlerpunkt
      v_schicht: "{{ states('input_select.schicht_auswahl') }}"
```

3. Bereinigter Zeitraumplaner v1.5

Die korrigierte Version führte die dynamische Jinja2-Weiche ein und beseitigte dadurch die Hidden-Dropdown-Problematik. ■filecite■turn4file0■

```
alias: "Schichtplaner: Zeitraum planen"

sequence:

  - variables:
```

```

v_person: "{{ states('input_select.planung_person') }}"

v_schicht: >-
  {% if v_person == 'Sascha' %}

    {{ states('input_select.planung_schichten_sascha') }}

  {% elif v_person == 'Sabine' %}

    {{ states('input_select.planung_schichten_sabine') }}

  {% else %}

    {{ states('input_select.schicht_auswahl') }}

  {% endif %}

```

4. Erweiterte Zeitlogik

```

Frühschicht -> 06:00 - 14:00
Spätschicht -> 14:00 - 22:00
Nachtschicht -> 22:00 - 06:00

F2 -> 06:15 - 13:15
F4 -> 06:45 - 14:00
F5 -> 08:00 - 14:00

v_start_final: >-
  {% if 'F2' in v_schicht_raw %}
    06:15:00

  {% elif 'F4' in v_schicht_raw %}
    06:45:00

  {% elif 'F5' in v_schicht_raw %}
    08:00:00

  {% elif 'Früh' in v_schicht_raw %}
    06:00:00

```

5. Schichtzeiten Autofill

```

alias: Schichtzeiten Autofill

triggers:
  - entity_id:
    - input_select.planung_schichten_sascha
    - input_select.planung_schichten_sabine
    trigger: state

conditions:
  - condition: template
    value_template: >
      {{ trigger.from_state is not none and
        trigger.to_state.state != trigger.from_state.state }}

```

6. Statistikscript v1.5

Die Statistik fragt Kalenderdaten direkt ab und schreibt die Werte in persistente input_text-Helfer. turn4file0

```

alias: "Schichtstatistik aktualisieren"

sequence:

  - action: calendar.get_events

    target:
      entity_id:
        - calendar.sascha
        - calendar.sabine
        - calendar.familie

    data:
      start_date_time: "2026-01-01T00:00:00"
      end_date_time: "2026-12-31T23:59:59"

    response_variable: cal

  - action: input_text.set_value

    target:
      entity_id: input_text.stats_sascha_fruehschicht

```

```
data:
  value: >-
    {{ s_ev
      | selectattr('summary', 'search', 'Frühschicht')
      | list | length | string }}
```

7. Wechsel von Textlogik zu Zeitlogik

Die ursprüngliche Textlogik erzeugte Probleme durch Emojis, Schreibweisen und Umlaute.

```
ALT:
selectattr('summary', 'search', 'Früh')

NEU:
selectattr('start', 'search', 'T06:00')
```

8. Nächtlicher Statistik-Sync

Diese Automatisierung stellt sicher, dass auch externe Kalenderänderungen erkannt werden. [filecite](#) [turn4file0](#)

```
alias: "Schichtstatistik: Tägliche Systemsynchronisation"

trigger:

- platform: time
  at: "03:00:00"

- platform: homeassistant
  event: start

action:

- action: script.schichtstatistik_aktualisieren
```

9. Mülltonnen-Planer

```
alias: "Mülltonnen: Rhythmus planen (ganztägig)"

sequence:

- repeat:
    count: 52

    sequence:

    - action: calendar.create_event

      target:
        entity_id: calendar.familie

      data:
        summary: "Mülltonne"
```

10. Lovelace Legacy Fehler v1.2

Ein fehlender Punkt im Entitynamen führte zu permanenten GUI-Timeouts. [filecite](#) [turn4file0](#)

```
# FEHLERHAFTER CODE

entity: input_select.planung_person
```

11. Lovelace Bereinigte Version v1.5

```
type: vertical-stack

cards:

- type: entities

  entities:
    - entity: input_select.planung_person
```

```

      name: "Wer wird geplant?"

- type: conditional

  conditions:
    - entity: input_select.planung_person
      state: "Sascha"

- type: button

  name: "Planung Speichern & Statistik triggern"

  tap_action:
    action: call-service
    service: script.schichtplaner_zeitraum_planen

```

12. Statistik Markdown Tabelle

```

type: markdown

title: "■ Schichtstatistik Gesamtjahr 2026"

content: >

| Schichtart | Sascha | Sabine |

| Frühschicht |
{{ states('input_text.stats_sascha_fruehschicht') | default('0') }} | - |

| Dienst Heurich |
- |
{{ states('input_text.stats_sabine_heurich') | default('0') }} |

```

13. Entwickler-Hinweise

Wichtige Regeln für zukünftige Erweiterungen:

- immer umlautfreie input_text Helfer nutzen
- neue Schichten zuerst im Backend anlegen
- danach Statistikscript erweitern
- anschließend Dashboard anpassen
- Versionsnummern sauber erhöhen
- Reload-Schutz aktiv lassen
- Delays nicht entfernen

14. Vollständige Helferübersicht

```

input_select.planung_person
input_select.planung_schichten_sascha
input_select.planung_schichten_sabine
input_select.schicht_auswahl

input_datetime.ziel_datum
input_datetime.ziel_datum_ende
input_datetime.start_zeit
input_datetime.end_zeit

input_text.stats_sascha_fruehschicht
input_text.stats_sascha_spaetschicht
input_text.stats_sascha_nachtschicht

input_text.stats_sabine_f2
input_text.stats_sabine_f4
input_text.stats_sabine_f5
input_text.stats_sabine_heurich

```

15. Finale Architektur-Erkenntnisse

Die YAML-Entwicklung zeigte: UI beeinflusst Backend massiv Hidden Dropdowns sind gefährlich Reloads erzeugen Seiteneffekte Zeitlogik ist stabiler als Textlogik Mischsysteme benötigen klare Datenquellen Versionierung ist extrem wichtig